Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Mini-C Compiler with Error Detection and Recovery System

Student / Team Information

Team Name: Team # (Mentor needs to assign)	Opticruze Compiler Design T099
Team member 1 (Team Lead)	Tomar, Ayushi -230223592 ayushitomar624@gmail.com

Team member 2	Laur, Shubham– 23021492 shubhamlaur9000@gmail.com
Team member 3	Baliyan, Yashika– 23022496 yashikabaliyan57@gmail.com
Team member 4	Tomar, Prashant- 230211589 prashanttomar1645@gmail.com

PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

This particular project is centered on the design and implementation of a compiler front-end component with a special focus on the techniques of lexical analysis and error handling. The main aim of this project is to develop a systematic and efficient lexical analyzer that is capable of scanning source code, detecting tokens, and reporting lexical errors along with effective error recovery strategies. This project is implemented using the Lex and Yacc tools, which are capable of automatically generating scanners and parsers to demonstrate basic concepts of compiler design.

The system is designed to identify different constructs of programming languages such as keywords, identifiers, operators, literals, delimiters, and comments. Moreover, the project places a special focus on effective error handling and identification, such as invalid tokens, incorrect identifiers, and illegal characters.

The project is divided into three main phases to ensure systematic development and equal contribution of all team members. Phase 1 involves environment setup, project planning, and implementation of the basic lexical analyzer. Phase 2 will involve extending the functionality of the project by incorporating syntax analysis using Yacc and advanced token handling. Phase 3 will involve advanced error handling, testing, optimization, and final integration of all components of the compiler.

The team-based approach will ensure equal contribution of all team members, including design, implementation, testing, and documentation tasks. This project will provide real-world experience in compiler development, automation tools, and systematic software development approaches, thereby enhancing knowledge in programming language processing and system-level software development.

Updated Project Approach and Architecture (2 pts)

The project adopts a modular and staged compiler front-end architecture, which emphasizes lexical analysis, syntax analysis, error recovery, and intermediate code generation. The project is being implemented using Lex for tokenization and Yacc for syntax analysis, aided by the C programming language for implementation and integration.

The project approach is as follows :

1. Requirement Analysis and Planning
   - Describe project scope and objectives.
   - List compiler phases to be developed.
   - Break down project work into three development stages.
   - Assign equal responsibilities to all team members.

2. System Design and Architecture Planning
   - Design modular front-end architecture for the compiler.
   - Specify data flow between preprocessing, lexical, syntax, and semantic phases.
   - Plan integration of intermediate representation generation.

3. Development Environment Setup

- Set up MSYS environment and GCC compiler.
- Install and configure Lex and Yacc utilities.
- Organize project workspace using Visual Studio Code.
- Verify toolchain setup with sample scanner programs.

4. Preprocessing Module Development
- Strip comments and unnecessary whitespace.
- Implement basic preprocessor directives.
- Prepare clean source code for lexical analysis.

5. Lexical Analyzer Development
- Design regular expressions for token recognition.
- Identify keywords, identifiers, literals, operators, and delimiters.
- Develop token classification and token stream generation.
- Support line number tracking for error reporting.

6. Lexical Error Detection and Recovery
- Detect illegal characters and invalid tokens.
- Develop recovery plans to resume scanning after errors.
- Produce informative and organized error messages.

7. Syntax Analyzer Development
- Specify grammar rules in Yacc.
- Check program syntax with context-free grammar.
- Develop syntax error detection and panic mode recovery.

8. Semantic Analysis Module
- Implement management of symbol tables.
- Carry out type checking and scope analysis.
- Identify semantic errors and print error messages.

9. Intermediate Representation Generation Module
- Offer user choice to produce IR in TAC, DAG, postfix, or AST form.
- Translate parsed code into chosen intermediate representation.

10. Testing and Validation Module
- Conduct unit testing for each module separately.
- Test the system with correct and incorrect input programs.
- Check the accuracy of error messages and outputs.

11. Documentation and Final Integration Module
- Prepare technical and user documentation.
- Integrate all modules into a single system.
- Optimize system performance and complete project tasks.

SYSTEM ARCHITECTURE :

The project uses a pipeline-based compiler front-end architecture in which the source program undergoes several stages of sequential processing.

1. Input Source Code
   - Program written in Mini-C language provided by user.
   - Serves as the main input to the compiler system.

2. Preprocessing Module
   - Comments and unnecessary spaces removed.
   - Basic preprocessing directives processed.
   - Clean input code for scanning obtained.

3. Lexical Analyzer
   - Source code scanned character by character.
   - Tokens created based on pattern matching rules.
   - Lexical errors detected and recovered from.
   - Line number tracking done.
   - Output: Token stream

4. Syntax Analyzer
   - Token stream obtained from lexical analyzer.
   - Program structure validated based on grammar rules.
   - Syntax errors detected and panic mode recovery done.
   - Output: Parsed structure of program

5. Semantic Analyzer
   - Symbol table managed.
   - Type checking and scope analysis done.
   - Logical and semantic errors detected.

6. Intermediate Representation Generator
   - Validated program translated to user-chosen IR representation:
   - Three Address Code (TAC)
   - Directed Acyclic Graph (DAG)
   - Postfix notation
   - Abstract Syntax Tree representation

7. Output Module
   - Error diagnostic messages displayed.
   - Intermediate representation output generated.

HIGH-LEVEL DIAGRAM :

Source Code

↓

Preprocessing

↓

Lexical Analyzer (Error Detection & Suitable Recovery Technique)

↓

Syntax Analyzer (Error Detection & Panic Mode Recovery)

↓

Semantic Analyzer (Error Detection and Logging)

↓

IR Generator (in choice of AST, DAG , Postfix , TAC)

↓

Output (IR)

Tasks Completed (7 pts)

Task Completed	Team Member
The development environment was successfully configured using MSYS, GCC compiler, Lex, and Yacc tools. Necessary libraries and dependencies were installed and tested to ensure compatibility with the Windows-based development setup	SHUBHAM LAUR
The development of the lexical analyzer. The analyzer was designed using Lex and is capable of scanning source code and identifying fundamental tokens such as keywords, identifiers, numeric literals, operators, delimiters, and whitespace. The module also implements line number tracking to provide accurate error diagnostics.	AYUSHI TOMAR
lexical error detection mechanisms have been implemented to identify invalid characters, malformed tokens, and unterminated constructs.	PRASHANT TOMAR
Basic recovery strategies have been incorporated to allow the analyzer to continue scanning after detecting errors rather than terminating execution.	YASHIKA BALIYAN & SHUBHAM LAUR
Documentation of token classification rules, regular expression design, and testing procedures.	AYUSHI TOMAR & PRASHANT TOMAR
Unit testing has been performed using multiple input programs to verify token recognition accuracy and robustness of error detection.	YASHIKA BALIYAN

Challenges/Roadblocks (7 pts)

The major challenges encountered during implementation were :

1. During the early stages of the project, the team faced a number of technical and conceptual issues while establishing the compiler development environment and designing the lexical analyzer. One of the main issues was establishing the development environment on a Windows system. These issues were overcome by performing rigorous troubleshooting, referring to the official documentation of the tools, and testing the environment setup using test scanner programs.

2. Another main issue was the proper design of regular expressions to identify tokens. The distinction between identifiers, keywords, numeric literals, and operators needed proper prioritization of rules to overcome conflicts and misidentification of tokens. The team overcame this issue by referring to the rules of lexical precedence and conducting rigorous testing with different input examples.

3. Handling the complexity of lexical error detection and recovery was also a challenge. This included the detection of ill-formed tokens, characters, and unterminated expressions, while ensuring that the analyzer proceeds with its processing. This was handled by the team through the implementation of structured error reporting and scanning rules that enable the analyzer to proceed with its processing upon encountering errors.

4. Handling line number tracking and generating helpful error messages was also a challenge. This was due to the need for coordination between scanning rules and input handling mechanisms. This was handled by the team by incorporating the line number tracking logic into the lexical analyzer and its validation through multi-line test inputs.

5. Coordination of the team and balancing task distribution among the team members was also a challenge. This was handled by the team through structured work allocation, progress discussions, and joint testing sessions.

6. Another major challenge was to identify various recovery techniques in different phases . This was handled by various source studying of different techniques .

7. Future challenges will be encountered in grammar implementation, parser integration, and intermediate representation. These will be handled by the team through the implementation of modular design, incremental development, and comprehensive testing at each stage to ensure system stability and correctness.

Tasks Pending (7 pts)

Task Pending	Team Member (to complete the task)
The development and integration of the syntax analyzer using Yacc.	AYUSHI TOMAR & SHUBHAM LAUR
Syntax error detection and recovery techniques, such as panic mode recovery.	YASHIKA BALIYAN & PRASHANT TOMAR
The development and the implementation of semantic analysis.	AYUSHI TOMAR
The detection of semantic errors such as undeclared variables, type mismatches, and invalid assignments while providing meaningful diagnostic messages.	SHUBHAM LAUR
Implementation of the intermediate representation (IR) generation module.	YASHIKA BALIYAN & PRASHANT TOMAR

Project Outcome/Deliverables (2 pts)

The project outcomes are stated below :

- Development of a functional compiler front-end for the Mini-C language.
- Development of a preprocessing module for comment removal and input code cleaning.
- Development and implementation of a lexical analyzer using Lex for: Identification of keywords, identifiers, operators, literals, and delimiters . Token stream generation . Detection and classification of lexical errors . Error recovery and line number tracking.
- Development of a syntax analyzer using Yacc for : Program structure validation based on grammar rules. Syntax error detection . Implementation of panic mode error recovery.
- Development of a semantic analysis module for: Symbol table management. Type checking. Validation of variable declarations and scope. Semantic error detection.

- Development of an Intermediate Representation (IR) Generator with output format options: Three Address Code (TAC) , Directed Acyclic Graph (DAG) , Postfix notation or Abstract Syntax Tree (AST) .
- Development of structured and meaningful error diagnostic messages.
- Development of complete project documentation including design, implementation, testing, and user manuals.

Progress Overview (2 pts)

The work is going as per the planned development schedule, and most of the tasks in Phase 1 have been completed. The team has been able to finalize project planning, requirement analysis, and system architecture design. The development environment has been set up, including the installation and testing of Lex, Yacc, MSYS, and GCC tools. The preprocessing module and basic lexical analyzer have been developed and tested using various input programs. Token recognition, token classification, and basic lexical error handling with line number tracking have been successfully accomplished.

Most of the Phase 1 tasks have been accomplished on schedule. The documentation work related to token definition, regular expression design, and testing has also been accomplished. This provides a robust start for the next phases of the project.

At present, grammar design for the syntax analyzer and sophisticated error recovery mechanisms are underway and need further polishing and testing. These need to be accomplished with meticulous rule ordering and verification for correct parsing actions.

The project is on schedule. There have been no significant delays so far. The team has been regularly coordinating and distributing their efforts evenly. With the basic modules already developed, the team is ready to move ahead with parser development, semantic analysis, and intermediate code generation in the next phases.

Codebase Information (2 pts)

Repository Link:
The source code of the project is stored in a centralized version-controlled repository for collaborative development, version management, and secure code merging.
Repository URL: [https://github.com/Ayushi-tomar02/compiler-Design]

Active Branch:
Main Branch: main – Holds the stable and tested code of the project.
Development Branch: phase1-development – Used for developing and testing Phase 1 modules like preprocessing and lexical analysis before code merging into the main branch.

Important Commits:

1. Initial Repository Setup Commit
2. Environment Configuration Commit
3. Preprocessor Module Implementation Commit
4. Lexical Analyzer Implementation Commit
5. Lexical Error Detection Commit
6. Testing and Debugging Commit

Testing and Validation Status (2 pts)

Test Type	Status (Pass/Fail)	Notes
Unit testing on individual modules.	PASS	Successfully passed
Integration testing to verify smooth interaction between modules.	PASS	Successfully passed
The system was tested with multiple datasets to check accuracy, efficiency, and reliability of results.	PASS	Successfully passed
Edge cases and invalid inputs were tested to ensure the system handles errors without crashing.	PASS	Successfully passed

Deliverables Progress (2 pts)

- Development of a functional compiler front-end for the Mini-C language. [COMPLETED]
- Development of a preprocessing module for comment removal and input code cleaning. [COMPLETED]
- Development and implementation of a lexical analyzer using Lex for: Identification of keywords, identifiers, operators, literals, and delimiters . Token stream generation . Detection and classification of lexical errors . Error recovery and line number tracking. [COMPLETED]
- Development of a syntax analyzer using Yacc for : Program structure validation based on grammar rules. Syntax error detection . Implementation of panic mode error recovery. [IN PROGRESS]
- Development of a semantic analysis module for: Symbol table management. Type checking. Validation of variable declarations and scope. Semantic error detection. [IN PROGRESS]
- Development of an Intermediate Representation (IR) Generator with output format options: Three Address Code (TAC) , Directed Acyclic Graph (DAG) , Postfix notation or Abstract Syntax Tree (AST) . [PENDING]
- Development of structured and meaningful error diagnostic messages. [PENDING]
- Development of complete project documentation including design, implementation, testing, and user manuals. [PENDING]